Studio Nx

JS 283 — Connect 칸과 연결 저장

2026-10-19

JavaScript 283 — Connect 칸과 연결 저장 (sequencer-devices.js)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`
작업	S1-10a 클립·연결·패턴 API: Connect 칸 그리기, 연결 저장 · 끄기, 트랙 줄 배지 (PRD 02 FR-2197 연결)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 282 클립 색인과 타임라인 클립, 다음 284 클립에 서버 번호 달기

1. 하는 일

한 트랙이 다른 트랙으로 소리를 보내는 연결(`track_connections`)을 읽어 두었다가, 펼침 칸의 CONNECT TO ANOTHER 칸에 셀렉트와 번호 칸으로 보여 줍니다. 값을 바꾸면 서버에 저장하고, 끄면 지웁니다. `track_connections` 표는 `UNIQUE(from_track_id)` 이므로 보내는 트랙마다 최대 한 건입니다.

```
GET /full 의 tracks[].connections
        ↓ indexConnections
  connections[from_track_id] = 연결 한 건
  connectionsByTarget[to_track_id] = [연결들]
        ↓
  connectHtml(track) — .sq-route (.sq-route__type · __track · __plugin)
        ↓ wireCells 의 input · change
  refillPluginOptions — 대상 트랙 번호가 바뀌면 이펙트 목록 다시 채움
  updateConnection(acc, track)
   ├ 끄기(None)   → DELETE /tracks/{id}/connection
   └ 켜기(Connected) → PUT  /tracks/{id}/connection { to_track_id, target, to_track_plugin_id? }
        ↓
  refreshConnections → →N · ⇠X 배지, Effects 칸의 sends to, Mixer 칸의 OFF 덮개 다시 그림
```

2. 언제 쓰나

쓰는 곳	
`applyFullProject`	`/full` 의 연결을 `indexConnections` 로 담습니다
`renderCells`	CONNECT TO ANOTHER 칸에 `connectHtml(track)` 을 그립니다
`wireCells` 의 `input`	Track 번호 칸이 바뀌면 `refillPluginOptions` 로 Plugin 목록을 다시 채웁니다
`wireCells` 의 `change`	셀렉트를 바꾸면 `is-live` 를 토글하고, 번호와 Plugin 값을 고르면 `updateConnection` 으로 서버에 저장합니다
`effectsHtml` · `mixerHtml` · `syncTrackSubs`	이펙트 사슬 끝의 `sendtoHtml`, 믹서 칸의 `offHtml`, 트랙 이름의 `connectionBadge` 를 붙입니다

3. 코드

```
/** 나가는 연결을 트랙별로 담기 (S1-10a FR-2197) — GET /full 의 각 트랙 connections 를 폅니다. */
function indexConnections(list) {
  connections = {};
  connectionsByTarget = {};
  (list || []).forEach(function (track) {
    const out = track.connections || [];
    if (out.length) connections[track.track_id] = out[0];
    out.forEach(function (c) {
      if (!connectionsByTarget[c.to_track_id]) connectionsByTarget[c.to_track_id] = [];
      connectionsByTarget[c.to_track_id].push(c);
    });
  });
}

/** 표시 번호(N) → 트랙 (MASTER 제외한 순서 기준) */
function trackByNumber(n) {
  const want = Number(n) - 1;
  if (!isFinite(want)) return null;
  for (let i = 0; i < tracks.length; i += 1) {
    if (tracks[i].track_type !== 'MASTER' && tracks[i].order_index === want) return tracks[i];
  }
  return null;
}

/** Connect to Another 칸 그리기 — 지금 연결 값을 셀렉트에 채웁니다 (None/Connected · Track · Plugin) */
function connectHtml(track) {
  const c = connections[track.track_id] || null;
  const on = !!c;
  return '<div class="sq-route' + (on ? ' is-live' : '') + '">'
    + '<label>Connect</label>'
    + '<select class="sq-route__type" aria-label="Connect">'
    + '<option value="none"' + (on ? '' : ' selected') + '>None</option>'
    + '<option value="connected"' + (on ? ' selected' : '') + '>Connected</option></select>'
    + '<label>Track</label>'
    + '<input class="sq-route__track" type="number" min="1" placeholder="—"'
    + (on ? ' value="' + trackNumberOf(c.to_track_id) + '"' : '') + '>'
    + '<label>Plugin</label>'
    + '<select class="sq-route__plugin"' + (on ? '' : ' disabled') + '>'
    + pluginOptionsHtml(on ? c.to_track_id : null, on ? c : null) + '</select>'
    + '</div>';
}

/** Plugin 셀렉트의 option 들 — 'Instruments (layer)' + 대상 트랙의 이펙트 (고른 것 표시) */
function pluginOptionsHtml(targetId, c) {
  const opts = [{ value: 'instruments', label: 'Instruments (layer)' }];
  const d = targetId ? devices[targetId] : null;
  if (d) {
    d.effects.forEach(function (e) {
      const p = pluginById(e.plugin_id);
      opts.push({ value: String(e.track_plugin_id), label: p ? p.name : 'Plugin #' + e.plugin_id });
    });
  } else if (c && c.to_track_plugin_id) {
    opts.push({ value: String(c.to_track_plugin_id), label: connectionTargetLabel(c) });
  }
```

```
  const picked = c && c.target === 'PLUGIN' && c.to_track_plugin_id ? String(c.to_track_plugin_id) : 'in
struments';
  return opts.map(function (o) {
    return '<option value="' + esc(o.value) + '"' + (o.value === picked ? ' selected' : '') + '>' + esc
(o.label) + '</option>';
  }).join('');
}

/** Track 번호가 바뀌면 그 트랙의 이펙트로 Plugin 칸을 다시 채움 (저장은 change 에서) */
function refillPluginOptions(route, track) {
  const num = route.querySelector('.sq-route__track');
  const plugin = route.querySelector('.sq-route__plugin');
  if (!num || !plugin) return;
  const target = trackByNumber(num.value);
  const conn = connections[track.track_id];
  const c = target && conn && conn.to_track_id === target.track_id ? conn : null;
  plugin.innerHTML = pluginOptionsHtml(target ? target.track_id : null, c);
}

/**
 * Connect to Another 값을 서버에 저장 (FR-2197) — Connected 면 PUT, None 이면 DELETE.
 * 부르는 곳: wireCells 의 change 처리. 저장 뒤 트랙 줄 배지를 다시 그립니다.
 */
export async function updateConnection(acc, track) {
  const route = acc.querySelector('.sq-route');
  if (!route) return;
  const type = route.querySelector('.sq-route__type');
  const num = route.querySelector('.sq-route__track');
  const plugin = route.querySelector('.sq-route__plugin');
  const on = !!type && type.value === 'connected' && !!num && num.value;
  if (!on) {
    try {
      await api.delete('/tracks/' + track.track_id + '/connection');
      delete connections[track.track_id];
      refreshConnections();
      notify({ type: 'connection', trackId: track.track_id, on: false });
    } catch (err) {
      if (!(err instanceof ApiError && err.status === 404)) warn(err);
    }
    return;
  }
  const target = trackByNumber(num.value);
  if (!target) { showToast('No track with that number'); return; }
  const value = plugin ? plugin.value : 'instruments';
  const instruments = !value || value === 'instruments';
  const body = {
    to_track_id: target.track_id,
    target: instruments ? 'INSTRUMENTS' : 'PLUGIN',
  };
  if (!instruments) body.to_track_plugin_id = Number(value);
  try {
    const saved = await api.put('/tracks/' + track.track_id + '/connection', body);
    connections[track.track_id] = saved;
    refreshConnections();
    notify({ type: 'connection', trackId: track.track_id, on: true, toTrackId: target.track_id });
  } catch (err) {
    warn(err);
```

```
    showToast(errText(err));
  }
}

/** 이펙트 칸 끝의 '→ sends to Track N › …' (FR-2197) */
function sendtoHtml(c) {
  const label = 'Track ' + trackNumberOf(c.to_track_id) + ' › ' + connectionTargetLabel(c);
  return '<li class="sq-sendto">→ sends to <b>' + esc(label) + '</b>'
    + '<span>then uses Track ' + trackNumberOf(c.to_track_id) + '’s effects and mixer</span></li>';
}

/** 믹서 칸 위 OFF 덮개 (FR-2197) — 보내는 트랙은 자기 이펙트·믹서 대신 받는 트랙 것을 씁니다 */
function offHtml(c) {
  return '<div class="sq-off"><b>OFF</b>Connected to<br>Track ' + trackNumberOf(c.to_track_id) + ' ›<br
>'
    + esc(connectionTargetLabel(c)) + '</div>';
}
```

문법 풀이

식	뜻
`connections[track.track_id] = out[0]`	`UNIQUE(from_track_id)` 이므로 첫 번째 곧 전부입니다
`isFinite(want)`	`Number('')` 는 0 이 되므로, 숫자가 아니면 여기서 null 을 돌려줍니다
`' sq-route' + (on ? ' is-live' : '')`	연결이 있으면 칸 전체에 `is-live` 를 붙여 셀렉트들을 살아 있게 합니다
`plugin.disabled = !on`	끈 상태에서는 Plugin 칸을 고를 수 없게 막습니다
`target: instruments ? 'INSTRUMENTS' : 'PLUGIN'`	악기 겹(레이어)이면 'INSTRUMENTS', 이펙트 하나이면 'PLUGIN' 입니다
`err.status === 404`	이미 끈 연결을 또 끌 때 서버가 404 를 주어도 화면에는 조용히 반영합니다
`setTimeout(..., 0)` (`wireCells`)	change 직후 DOM 값이 아직 바뀌기 전이므로 다음 틱에 읽어 저장합니다

4. 왜 이렇게

선택	이유
셀렉트 값이 `none` 과 `connected` 두 가지	지금 저장할 수 있는 상태는 끈 것과 건 것뿐입니다. sendto 는 대상 트랙이 이미 다른 곳으로 보내는 경우의 표시이므로 `sendtoHtml` 과 `offHtml` 로만 보여 주고 선택값으로는 두지 않았습니다
하나의 연결만 허용	DB 의 `UNIQUE(from_track_id)` 가 마지막으로 지키므로, 같은 트랙에서 다시 저장하면 `PUT` 이 기존 행을 고치는 방식이 됩니다
Track 칸은 번호, Plugin 칸은 셀렉트	사용자가 트랙 이름을 외우지 않아도 화면의 `1.`, `2.` 번호를 그대로 쓸 수 있습니다
`refillPluginOptions` 는 input 에서, 저장은 change 에서	번호를 칠 때마다 저장하면 서버 요청이 너무 많습니다. 목록만 미리 맞추고 저장은 고침이 끝난 뒤에 합니다
보내는 트랙의 믹서에 OFF 덮개	실제로 믹서를 쓰는 쪽은 받는 트랙입니다. 화면이 실제로 쓰지 않는 볼륨 칸을 그대로 두면 잘못 만질 수 있습니다

선택	이유
`notify({ type: 'connection', … })`	연결도 저장 · 이력의 대상이므로 장치 변경과 같은 알림 경로를 씁니다 (273 문서의 `setDeviceNotifier`)

5. 시험

`docs/handoff/s1-10a_tests/ui.mjs` 에서 트랙의 CONNECT 칸에서 Connected 를 고르고 Track 번호를 넣은 뒤 페이지를 새로 불러와도 연결이 남는지, Effects 칸 끝에 sends to 문구가 붙는지, 대상 트랙 믹서에 OFF 덮개가 생기는지, None 으로 되돌리면 `DELETE` 후 연결이 사라지는지 확인합니다. 시험은 14개 항목 모두 통과했고, 화면 사진은 `docs/screenshots/2026-10-19_s1-10a_clips_connections/02_connection_saved.png` 에 있습니다.